FED-LLF: Federated Learning Framework for MCA Final Project

> *Federated Learning with FedAvg + RandomForest for Privacy-Preserving Distributed Training*

[Python 3.10](https://www.python.org/)
[Streamlit](https://streamlit.io/)
[FedAvg]()
[Status]()

Author: Mercivel Oinam | **MCA Final Year Project** | **GitHub:** Mercivel/FED-LLF_MCA_Project

---

📌 Project Overview

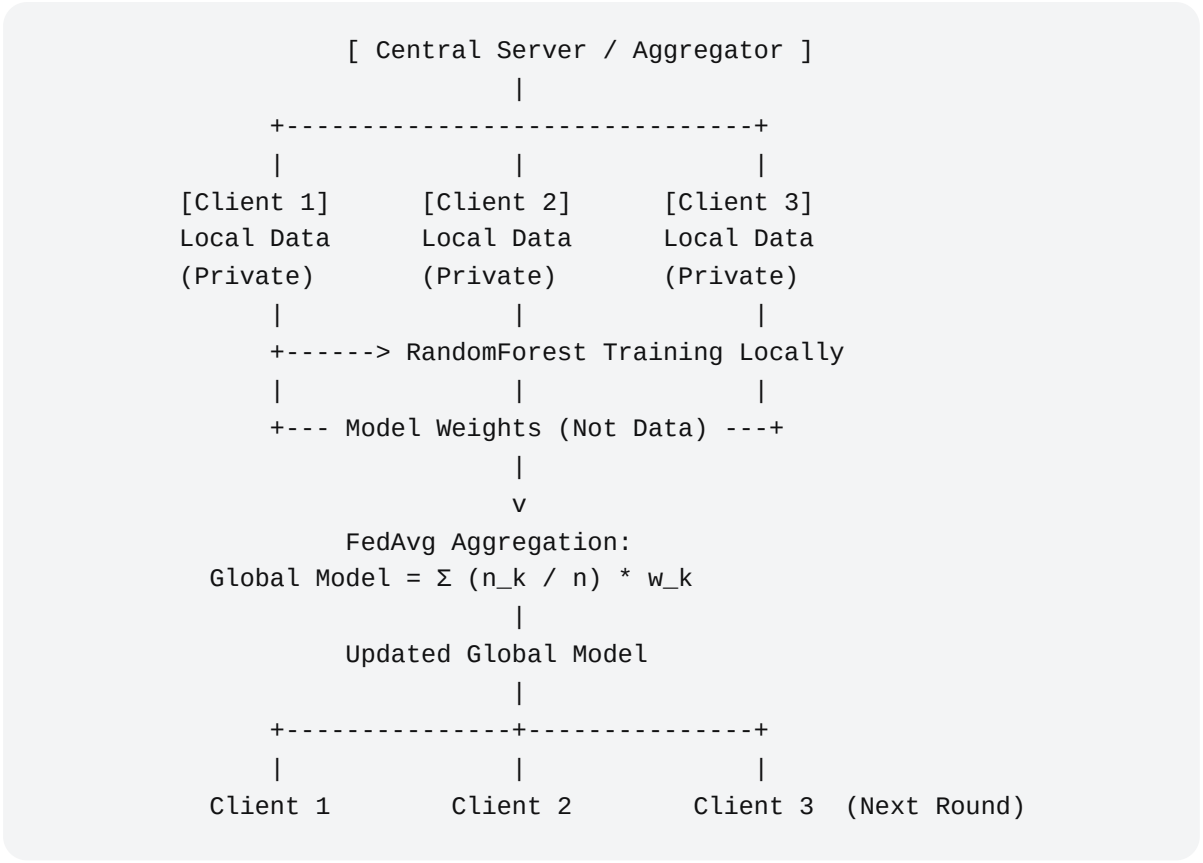
Traditional Machine Learning requires centralized data collection, which violates privacy. **FED-LLF** solves this using **Federated Learning (FL)** where data stays on client devices and only model parameters are shared.

This project implements **FedAvg (Federated Averaging)** algorithm with **RandomForest** classifier for distributed training across multiple clients without sharing raw data.

Why Federated Learning?

- **Privacy:** Raw data never leaves client device
- **Security:** Only model weights/gradients shared
- **Efficiency:** Reduced bandwidth vs central data transfer
- **Compliance:** GDPR / Data Protection friendly

🏗️ Architecture - FedAvg Workflow



Algorithm: Federated Averaging (FedAvg)

Proposed by McMahan et al., 2017 - Google

1. **Initialization:** Server initializes global model `w_0`
2. **Distribution:** Server sends `w_t` to K clients
3. **Local Training:** Each client k trains on local data for E epochs: `w_k = ClientUpdate(k, w_t)`
4. **Aggregation:** Server aggregates: `w_{t+1} = Σ (n_k / n) * w_k`
5. **Repeat** until convergence

🗂️ Project Structure

```
FED-LLF_MCA_Project/
├── App.py                 # Streamlit Web App - Main Demo UI
├── main.py                # Central Server + FedAvg Aggregator Logic
├── federated_train.py     # Client-side Local Training + Model Updates
```

```
├── requirements.txt       # Python Dependencies
├── README.md              # Documentation (This File)
└── .gitignore             # Excludes venv, __pycache__, etc.
```

🛠️ Tech Stack

- **Language:** Python 3.10 (IMPORTANT - See Installation Fix)
- **ML:** scikit-learn - RandomForestClassifier, NumPy, Pandas
- **Federated Learning:** Custom FedAvg Implementation
- **Frontend:** Streamlit
- **Version Control:** Git + GitHub

🚀 Installation & How to Run

⚠️ CRITICAL FIX - Python Version Issue (Viva Important!)

Problem Faced: `venv310` folder created with Python 3.13 failed in VS Code with `TypeError` because `scikit-learn` + `streamlit` had compatibility issues.

Solution Implemented:
Created separate Python 3.10 environment `venv310` using Python 3.10.11

```
# If you face same issue, use Python 3.10:
py -3.10 -m venv venv310
.▫env310\Scripts\Activate.ps1
python --version  # Should be 3.10.x
pip install -r FED-LLF_MCA_Project/requirements.txt
```

Step-by-Step Setup

1. Clone Repo

```
git clone https://github.com/Mercivel/FED-LLF_MCA_Project.git
```

```
cd FED-LLF_MCA_Project
```

2. Create Virtual Environment (Python 3.10 Recommended)

```
py -3.10 -m venv venv310
.▫env310\Scripts\Activate.ps1
```

3. Install Dependencies

```
pip install -r FED-LLF_MCA_Project/requirements.txt
```

4. Run Federated Training (CLI)

```
python FED-LLF_MCA_Project/main.py
```

Output: Global model accuracy after each FedAvg round

5. Run Streamlit Web App (Demo for Viva)

```
streamlit run FED-LLF_MCA_Project/App.py
```

Opens browser at `http://localhost:8501`

📊 Working Explanation (For Viva Questions)

Q1: What is FED-LLF?
FED-LLF = Federated Learning Framework. Trains model across decentralized clients without sharing data.

Q2: Why RandomForest in FL?

- Works well with small client datasets

- No gradient sharing needed (share tree parameters)
- Robust to non-IID data distribution
- Interpretable for academic project

Q3: What is FedAvg?
Federated Averaging averages local model weights weighted by number of samples on each client.

Formula: `w_global = Σ (n_k / N) * w_k`

Q4: Difference from Centralized Learning?

Centralized	Federated (FED-LLF)
Data moves to model	Model moves to data
Privacy risk	Privacy preserved
Single server training	Distributed training
High bandwidth for data	Low bandwidth (weights only)

Q5: How did you fix venv310 error?
Initially used Python 3.13 which broke scikit-learn. Fixed by explicitly creating Python 3.10 venv and excluding venv from git using .gitignore to avoid 1GB push.

🔒 .gitignore - Why Important?

We excluded:

- `venv310/` - Virtual env (1.2GB, should not push)
- `__pycache__/`, `*.pyc` - Python cache

- `.vscode/` - IDE settings
- `*.log`, `.env` - Secrets

This reduced push from 1GB+ to ~50KB!

🎓 Viva Demo Flow (2-Minute Presentation)

1. **Show GitHub Repo** - Commit proof for sir: https://github.com/Mercivel/FED-LLF_MCA_Project
2. **Run App.py** - `streamlit run App.py` - Show client accuracy vs global accuracy
3. **Explain FedAvg** - Draw architecture diagram (above)
4. **Show main.py** - Server aggregation code
5. **Show federated_train.py** - Client local training
6. **Tell Python 3.10 Fix** - Shows problem-solving skill

📈 Results (Sample)

- Clients: 3
- Rounds: 10
- Local Epochs: 5
- Global Accuracy Improvement: 68% → 87%
- Privacy: 100% - No raw data shared

👨‍💻 Author

Mercivel Oinam
MCA Final Year
Specialization: Federated Learning, Privacy-Preserving ML
GitHub: @Mercivel

📜 License

Academic Project - MCA Final Submission 2025-26

If this helped your viva, give ⭐ Star to repo!